

캠퍼스 커리어 플래너 4차 과제를 해야 해. 기존 3차 코드에서 메뉴, 함수, 전역변수를 사용했는데, 이번에는 이중 리스트, 파일 저장, 예외처리까지 넣어야 해. 너무 어려운 기능은 넣지 말고, 과제 조건에 맞게 V4.0 구조를 어떻게 잡으면 좋을지 정리해줘.

9s 동안 생각함

좋아. 4차 V4.0은 기존 3차 메뉴형 프로그램을 유지하면서, 데이터 누적 저장과 파일 저장, 예외처리만 추가하는 방향이 제일 안전해. 4차 PDF 핵심 조건도 이중 리스트, append 누적, 반복 출력, 파일 저장, try-except 2곳 이상이야.

V4.0 구조 추천

1. 전역 이중 리스트 유지

```
task_list = []
```

일정 하나를 리스트로 만들고, 그걸 task_list에 추가하면 이중 리스트가 됨.

```
task = [user_name, task_name, category, days_left, importance, hours, score, grade]
task_list.append(task)
```

즉 구조는 이런 느낌:

```
[
    ["박성연", "기말 공부", "학교공부", 3, 5, 4, 55, "S"],
    ["박성연", "인턴 지원", "취업", 2, 5, 3, 54, "S"]
]
```

2. 메뉴 구조

기존 3차 메뉴에서 파일 저장만 추가하면 충분해.

1. 일정 입력
2. 전체 일정 조회
3. 최근 일정 긴급도 분석
4. 파일 저장
5. 종료

너무 복잡하게 파일 불러오기까지 넣을 필요는 없어.

파일 저장만 해도 PDF의 파일 저장 조건은 충족 가능해.

3. 함수 구성

기존 함수 구조 유지하면 됨.

```
show_menu()
add_task()
calculate_urgency()
decide_grade()
show_tasks()
analyze_task()
save_file()
```

이 정도면 충분하고, 함수/메뉴 구조도 유지됨.

4. 예외처리 넣을 위치

최소 2곳 넣어야 하니까 아래 두 군데가 제일 자연스러움.

예외처리 1: 숫자 입력 오류

일정 입력할 때 문자 입력하면 프로그램이 꺼지지 않게 처리.

try:

```
category_num = int(input("번호 입력: "))
days_left = int(input("마감까지 남은 일수 입력: "))
importance = int(input("중요도 입력: "))
hours = float(input("예상 소요 시간 입력: "))
```

except ValueError:

```
    print("숫자를 입력해야 합니다.")
```

예외처리 2: 파일 저장 오류

파일 저장할 때 문제가 생기면 안내 메시지 출력.

try:

```
    with open("career_tasks.txt", "w", encoding="utf-8") as file:
        ...
```

except:

```
    print("파일 저장 중 오류가 발생했습니다.")
```

최종 방향 한 줄

3차 코드에서 메뉴와 함수 구조는 그대로 두고, 여러 일정을 이중 리스트에 누적 저장하도록 바꾸고, 전체 일정 출력과 파일 저장, try-except 예외처리만 추가하면 된다.

이 정도면 기본에 충실하고, 과제 조건에도 딱 맞아.

좋아. 그러면 기존 캠퍼스 커리어 플래너에 여러 일정을 저장하는 이중 리스트 구조를 어떻게 적용하면 좋을지 알려줘. 일정 입력, 전체 조회, 긴급도 분석, 파일 저장 메뉴 정도만 넣고 싶어.

9s 동안 생각함

좋아. 이중 리스트는 “일정 1개 = 작은 리스트, 전체 일정 목록 = 큰 리스트”로 생각하면 돼.

4차 과제에서 요구하는 것도 여러 항목 데이터를 이중 리스트에 append로 누적하고, for 문으로 출력하고, 파일로 저장하는 구조야.

이중 리스트 적용 방식

전역변수로 전체 일정 저장 리스트를 둔다.

```
task_list = []
```

일정 하나를 입력받으면 이렇게 하나의 리스트로 만든다.

```
task = [user_name, task_name, category, days_left, importance, hours, score, grade]
```

그다음 전체 리스트에 추가한다.

```
task_list.append(task)
```

그러면 실제 구조는 이렇게 됨.

```
task_list = [
    ["박성연", "기말 공부", "학교공부", 3, 5, 4.0, 55.0, "S"],
    ["박성연", "인턴 지원", "취업", 2, 5, 3.0, 54.0, "S"],
    ["박성연", "토익 공부", "자격증", 10, 3, 2.0, 24.0, "C"]
]
```

메뉴는 이 정도면 충분함

1. 일정 입력
2. 전체 일정 조회
3. 최근 일정 긴급도 분석

4. 파일 저장
5. 종료

각 메뉴 역할

1. 일정 입력

사용자가 이름, 할 일, 분야, 기한, 중요도, 예상 시간을 입력하면 긴급도 점수와 등급을 계산해서 task_list에 추가.

2. 전체 일정 조회

for문으로 task_list 안의 여러 일정을 차례대로 출력.

이게 4차 과제의 "이중 리스트 출력" 조건에 해당함.

3. 최근 일정 긴급도 분석

가장 마지막에 입력한 일정 task_list[-1]을 기준으로 긴급도 점수, 등급, 특별 분류 메시지를 출력.

4. 파일 저장

with open("career_tasks.txt", "w", encoding="utf-8")를 사용해서 task_list 안의 일정들을 텍스트 파일로 저장.

5. 종료

break로 프로그램 종료.

예외처리는 여기 넣으면 자연스러움

첫 번째는 일정 입력에서 숫자를 잘못 입력했을 때:

try:

```
days_left = int(input("마감까지 남은 일수 입력: "))
```

```
importance = int(input("중요도 입력: "))
```

```
hours = float(input("예상 소요 시간 입력: "))
```

except ValueError:

```
print("숫자를 입력해야 합니다.")
```

두 번째는 파일 저장할 때:

try:

```
with open("career_tasks.txt", "w", encoding="utf-8") as file:
```

```
...
```

except:

```
print("파일 저장 중 오류가 발생했습니다.")
```

정리하면, 3차 코드의 기본 메뉴/함수 구조는 유지하고, task_list에 여러 개의 task를 누적 저장하는 방식으로 바꾸면 4차 과제 조건에 딱 맞아.

그 구조를 바탕으로 실제 main.py 코드를 작성해줘. 이중 리스트, append, for문 출력, 파일 저장, try-except 2곳 이상이 들어가게 해줘. 다른 언급하지 않은, 배우지 않은 고급 문법은 쓰지 말고 파이썬 입문 수준으로 작성해줘.

37s 동안 생각함

좋아. 4차 조건에 맞게 **기본에 충실한 V4.0 코드**로 짰어.

이중 리스트, append, for문 출력, 파일 저장, try-except 2곳 이상 들어가 있음. 4차 PDF 평가조건에도 이 항목들이 핵심으로 들어가 있어.

파일이름 : 캠퍼스 커리어 플래너 V4.0

작 성 자 : 60211865 박성연

```
task_list = []
```

```
def show_menu():
```

```
    print("\n===== 캠퍼스 커리어 플래너 V4.0 =====")
```

```
    print("1. 일정 입력")
```

```
    print("2. 전체 일정 조회")
```

```
    print("3. 최근 일정 긴급도 분석")
```

```
    print("4. 파일 저장")
```

```
    print("5. 종료")
```

```
def calculate_urgency(importance, hours, days_left):
```

```
    score = (importance * 10) + (hours * 2) - days_left
```

```
    return score
```

```
def decide_grade(score):
```

```
    if score >= 50:
```

```
        grade = "S"
```

```
    elif score >= 40:
```

```
        grade = "A"
```

```
    elif score >= 30:
```

```
        grade = "B"
```

```
    elif score >= 20:
```

```
    grade = "C"
else:
    grade = "D"
return grade
```

```
def get_category(category_num):
    if category_num == 1:
        category = "학교공부"
    elif category_num == 2:
        category = "연구"
    elif category_num == 3:
        category = "취업"
    elif category_num == 4:
        category = "자격증"
    else:
        category = "잘못입력"
    return category
```

```
def add_task():
    global task_list

    print("\n[일정 입력]")
    user_name = input("이름 입력: ")
    task_name = input("할 일 입력: ")
```

```
    print("\n분야 선택")
    print("1. 학교공부")
    print("2. 연구")
    print("3. 취업")
    print("4. 자격증")
```

```
    try:
        category_num = int(input("번호 입력: "))
        days_left = int(input("마감까지 남은 일수 입력: "))
        importance = int(input("중요도(1~5) 입력: "))
```

```
    hours = float(input("예상 소요 시간 입력: "))
except ValueError:
    print("숫자를 입력해야 하는 곳에 문자를 입력했습니다.")
    return
```

```
category = get_category(category_num)
```

```
if category == "잘못입력" or importance < 1 or importance > 5 or days_left < 0 or
hours < 0:
    print("입력값이 올바르지 않습니다.")
    return
```

```
score = calculate_urgency(importance, hours, days_left)
grade = decide_grade(score)
```

```
task = [user_name, task_name, category, days_left, importance, hours, score, grade]
task_list.append(task)
```

```
print("일정이 저장되었습니다.")
```

```
def show_tasks():
    print("\n[전체 일정 조회]")
```

```
if len(task_list) == 0:
    print("저장된 일정이 없습니다.")
```

```
else:
    count = 1
```

```
for task in task_list:
    print("\n-----", count, "번째 일정 -----")
    print("이름:", task[0])
    print("할 일:", task[1])
    print("분야:", task[2])
    print("남은 기한:", task[3], "일")
    print("중요도:", task[4])
```

```
print("예상 소요 시간:", task[5], "시간")
print("긴급도 점수:", task[6])
print("우선순위 등급:", task[7])
count = count + 1
```

```
def analyze_recent_task():
```

```
    print("\n[최근 일정 긴급도 분석]")
```

```
    if len(task_list) == 0:
```

```
        print("분석할 일정이 없습니다.")
```

```
    else:
```

```
        task = task_list[-1]
```

```
        category = task[2]
```

```
        days_left = task[3]
```

```
        importance = task[4]
```

```
        hours = task[5]
```

```
        score = task[6]
```

```
        grade = task[7]
```

```
        if (category == "취업" and days_left <= 3 and importance >= 4) or (category == "연구" and importance == 5):
```

```
            special_title = "최우선 집중 일정"
```

```
        elif category == "자격증" and (days_left <= 7 or hours >= 5):
```

```
            special_title = "단기 관리 필요 일정"
```

```
        elif category == "학교공부" and importance >= 4 and hours >= 4:
```

```
            special_title = "학습 집중 일정"
```

```
        else:
```

```
            special_title = "일반 일정"
```

```
    if grade == "S" or grade == "A":
```

```
        if category == "취업":
```

```
            message = "가장 먼저 확인해야 할 채용/인턴 일정"
```

```
        elif category == "연구":
```

```
            message = "연구 우선순위가 매우 높음"
```



```

elif category == "학교공부":
    message = "학업 일정을 우선적으로 처리해야 함"
else:
    message = "자격증 준비를 빠르게 진행"
else:
    if days_left <= 3:
        message = "등급은 높지 않지만 마감이 가까우니 주의 필요"
    else:
        message = "계획적으로 진행하면 되는 일정"

```

```

print("긴급도 점수:", score)
print("우선순위 등급:", grade)
print("특별 분류:", special_title)
print("안내 메시지:", message)

```

```

def save_tasks():
    print("\n[파일 저장]")

    if len(task_list) == 0:
        print("저장할 일정이 없습니다.")
    else:
        try:
            file = open("career_tasks.txt", "w", encoding="utf-8")

            file.write("이름,할일,분야,남은기한,중요도,예상소요시간,긴급도점수,등급\n")

            for task in task_list:
                line = str(task[0]) + "," + str(task[1]) + "," + str(task[2]) + "," + str(task[3]) + "," + str(task[4]) + "," + str(task[5]) + "," + str(task[6]) + "," + str(task[7]) + "\n"
                file.write(line)

            file.close()
            print("career_tasks.txt 파일 저장 완료")

        except:

```

```
print("파일 저장 중 오류가 발생했습니다.")
```

```
while True:
```

```
    show_menu()
```

```
    try:
```

```
        choice = int(input("메뉴를 선택하세요: "))
```

```
    except ValueError:
```

```
        print("메뉴는 숫자로 입력해야 합니다.")
```

```
        continue
```

```
    if choice == 1:
```

```
        add_task()
```

```
    elif choice == 2:
```

```
        show_tasks()
```

```
    elif choice == 3:
```

```
        analyze_recent_task()
```

```
    elif choice == 4:
```

```
        save_tasks()
```

```
    elif choice == 5:
```

```
        print("프로그램을 종료합니다.")
```

```
        break
```

```
    else:
```

```
        print("잘못된 메뉴입니다. 다시 선택하세요.")
```

이 코드는 **파일 저장 메뉴를 실행하면 career_tasks.txt가 생성**돼. 4차 제출물에 저장된 텍스트 파일도 관련 제출물로 적혀 있으니까, 실행 후 생성된 career_tasks.txt도 GitHub에 같이 올리면 안전해.